

15. Statements

C# provides a variety of statements. [Note: Most of these statements will be familiar to developers who have programmed in C and C++. *end note*]

```

statement:
    labeled-statement
    declaration-statement
    embedded-statement

embedded-statement:
    block
    empty-statement
    expression-statement
    selection-statement
    iteration-statement
    jump-statement
    try-statement
    checked-statement
    unchecked-statement
    lock-statement
    using-statement

```

The *embedded-statement* nonterminal is used for statements that appear within other statements. The use of *embedded-statement* rather than *statement* excludes the use of declaration statements and labeled statements in these contexts. [Example: The code

```

void F(bool b) {
    if (b)
        int i = 44;
}

```

results in a compile-time error because an `if` statement requires an *embedded-statement* rather than a *statement* for its if branch. If this code were permitted, then the variable `i` would be declared, but it could never be used. (Note, however, that by placing `i`'s declaration in a block, the example is valid.) *end example*]

15.1 End points and reachability

Every statement has an **end point**. In intuitive terms, the end point of a statement is the location that immediately follows the statement. The execution rules for composite statements (statements that contain embedded statements) specify the action that is taken when control reaches the end point of an embedded statement. For example, when control reaches the end point of a statement in a block, control is transferred to the next statement in the block.

If a statement can possibly be reached by execution, the statement is said to be **reachable**. Conversely, if there is no possibility that a statement will be executed, the statement is said to be **unreachable**.

[Example: In the example

```

void F() {
    Console.WriteLine("reachable");
    goto Label;
    Console.WriteLine("unreachable");
Label:
    Console.WriteLine("reachable");
}

```

the second invocation of `Console.WriteLine` is unreachable because there is no possibility that the statement will be executed. *end example*]

A warning is reported if the compiler determines that a statement is unreachable. It is specifically not an error for a statement to be unreachable.

[*Note:* To determine whether a particular statement or end point is reachable, the compiler performs flow analysis according to the reachability rules defined for each statement. The flow analysis takes into account the values of constant expressions (§14.15) that control the behavior of statements, but the possible values of non-constant expressions are not considered. In other words, for purposes of control flow analysis, a non-constant expression of a given type is considered to have any possible value of that type.

In the example

```
void F() {
    const int i = 1;
    if (i == 2) Console.WriteLine("unreachable");
}
```

the boolean expression of the `if` statement is a constant expression because both operands of the `==` operator are constants. As the constant expression is evaluated at compile-time, producing the value `false`, the `Console.WriteLine` invocation is considered unreachable. However, if `i` is changed to be a local variable

```
void F() {
    int i = 1;
    if (i == 2) Console.WriteLine("reachable");
}
```

the `Console.WriteLine` invocation is considered reachable, even though, in reality, it will never be executed. *end note*]

The *block* of a function member is always considered reachable. By successively evaluating the reachability rules of each statement in a block, the reachability of any given statement can be determined.

[*Example:* In the example

```
void F(int x) {
    Console.WriteLine("start");
    if (x < 0) Console.WriteLine("negative");
}
```

the reachability of the second `Console.WriteLine` is determined as follows:

- The first `Console.WriteLine` expression statement is reachable because the block of the `F` method is reachable (§15.2).
- The end point of the first `Console.WriteLine` expression statement is reachable §15.2 because that statement is reachable (§15.6 and §15.2).
- The `if` statement is reachable because the end point of the first `Console.WriteLine` expression statement is reachable (§15.6 and §15.2).
- The second `Console.WriteLine` expression statement is reachable because the boolean expression of the `if` statement does not have the constant value `false`.

end example]

There are two situations in which it is a compile-time error for the end point of a statement to be reachable:

- Because the `switch` statement does not permit a switch section to “fall through” to the next switch section, it is a compile-time error for the end point of the statement list of a switch section to be reachable. If this error occurs, it is typically an indication that a `break` statement is missing.
- It is a compile-time error for the end point of the block of a function member that computes a value to be reachable. If this error occurs, it typically is an indication that a `return` statement is missing.

15.2 Blocks

A *block* permits multiple statements to be written in contexts where a single statement is allowed.

```
block:
    { statement-listopt }
```

A *block* consists of an optional *statement-list* (§15.2.1), enclosed in braces. If the statement list is omitted, the block is said to be empty.

A block may contain declaration statements (§15.5). The scope of a local variable or constant declared in a block is the block.

Within a block, the meaning of a name used in an expression context must always be the same (§14.5.2.1).

A block is executed as follows:

- If the block is empty, control is transferred to the end point of the block.
- If the block is not empty, control is transferred to the statement list. When and if control reaches the end point of the statement list, control is transferred to the end point of the block.

The statement list of a block is reachable if the block itself is reachable.

The end point of a block is reachable if the block is empty or if the end point of the statement list is reachable.

15.2.1 Statement lists

A *statement list* consists of one or more statements written in sequence. Statement lists occur in *blocks* (§15.2) and in *switch-blocks* (§15.7.2).

```
statement-list:
    statement
    statement-list statement
```

A statement list is executed by transferring control to the first statement. When and if control reaches the end point of a statement, control is transferred to the next statement. When and if control reaches the end point of the last statement, control is transferred to the end point of the statement list.

A statement in a statement list is reachable if at least one of the following is true:

- The statement is the first statement and the statement list itself is reachable.
- The end point of the preceding statement is reachable.
- The statement is a labeled statement and the label is referenced by a reachable `goto` statement.

The end point of a statement list is reachable if the end point of the last statement in the list is reachable.

15.3 The empty statement

An *empty-statement* does nothing.

```
empty-statement:
    ;
```

An empty statement is used when there are no operations to perform in a context where a statement is required.

Execution of an empty statement simply transfers control to the end point of the statement. Thus, the end point of an empty statement is reachable if the empty statement is reachable.

[*Example:* An empty statement can be used when writing a `while` statement with a null body:

```
bool ProcessMessage() {...}
void ProcessMessages() {
    while (ProcessMessage())
        ;
}
```

Also, an empty statement can be used to declare a label just before the closing “`}`” of a block:

```
void F() {
    ...
    if (done) goto exit;
    ...
    exit: ;
}
```

end example]

15.4 Labeled statements

A *labeled-statement* permits a statement to be prefixed by a label. Labeled statements are permitted in blocks, but are not permitted as embedded statements.

labeled-statement:
identifier : *statement*

A labeled statement declares a label with the name given by the *identifier*. The scope of a label is the whole block in which the label is declared, including any nested blocks. It is a compile-time error for two labels with the same name to have overlapping scopes.

A label can be referenced from `goto` statements (§15.9.3) within the scope of the label. [*Note:* This means that `goto` statements can transfer control within blocks and out of blocks, but never into blocks. *end note*]

Labels have their own declaration space and do not interfere with other identifiers. [*Example:* The example

```
int F(int x) {
    if (x >= 0) goto x;
    x = -x;
    x: return x;
}
```

is valid and uses the name `x` as both a parameter and a label. *end example]*

Execution of a labeled statement corresponds exactly to execution of the statement following the label.

In addition to the reachability provided by normal flow of control, a labeled statement is reachable if the label is referenced by a reachable `goto` statement. (Exception: If a `goto` statement is inside a `try` that includes a `finally` block, and the labeled statement is outside the `try`, and the end point of the `finally` block is unreachable, then the labeled statement is not reachable from that `goto` statement.)

15.5 Declaration statements

A *declaration-statement* declares a local variable or constant. Declaration statements are permitted in blocks, but are not permitted as embedded statements.

declaration-statement:
local-variable-declaration ;
local-constant-declaration ;

15.5.1 Local variable declarations

A *local-variable-declaration* declares one or more local variables.

```

1      local-variable-declaration:
2          type local-variable-declarators

3      local-variable-declarators:
4          local-variable-declarator
5          local-variable-declarators , local-variable-declarator

6      local-variable-declarator:
7          identifier
8          identifier = local-variable-initializer

9      local-variable-initializer:
10         expression
11         array-initializer

```

The *type* of a *local-variable-declaration* specifies the type of the variables introduced by the declaration. The type is followed by a list of *local-variable-declarators*, each of which introduces a new variable. A *local-variable-declarator* consists of an *identifier* that names the variable, optionally followed by an “=” token and a *local-variable-initializer* that gives the initial value of the variable.

The value of a local variable is obtained in an expression using a *simple-name* (§14.5.2), and the value of a local variable is modified using an *assignment* (§14.13). A local variable must be definitely assigned (§12.3) at each location where its value is obtained.

The scope of a local variable declared in a *local-variable-declaration* is the block in which the declaration occurs. It is an error to refer to a local variable in a textual position that precedes the *local-variable-declarator* of the local variable. Within the scope of a local variable, it is a compile-time error to declare another local variable or constant with the same name.

A local variable declaration that declares multiple variables is equivalent to multiple declarations of single variables with the same type. Furthermore, a variable initializer in a local variable declaration corresponds exactly to an assignment statement that is inserted immediately after the declaration.

[*Example*: The example

```

27      void F() {
28          int x = 1, y, z = x * 2;
29      }

```

corresponds exactly to

```

31      void F() {
32          int x; x = 1;
33          int y;
34          int z; z = x * 2;
35      }

```

end example]

15.5.2 Local constant declarations

A *local-constant-declaration* declares one or more local constants.

```

39      local-constant-declaration:
40          const type constant-declarators

41      constant-declarators:
42          constant-declarator
43          constant-declarators , constant-declarator

44      constant-declarator:
45          identifier = constant-expression

```

The *type* of a *local-constant-declaration* specifies the type of the constants introduced by the declaration.

The type is followed by a list of *constant-declarators*, each of which introduces a new constant. A *constant-*

declarator consists of an *identifier* that names the constant, followed by an “=” token, followed by a *constant-expression* (§14.15) that gives the value of the constant.

The *type* and *constant-expression* of a local constant declaration must follow the same rules as those of a constant member declaration (§17.3).

The value of a local constant is obtained in an expression using a *simple-name* (§14.5.2).

The scope of a local constant is the block in which the declaration occurs. It is an error to refer to a local constant in a textual position that precedes its *constant-declarator*. Within the scope of a local constant, it is a compile-time error to declare another local variable or constant with the same name.

A local constant declaration that declares multiple constants is equivalent to multiple declarations of single constants with the same type.

15.6 Expression statements

An *expression-statement* evaluates a given expression. The value computed by the expression, if any, is discarded.

expression-statement:
statement-expression ;

statement-expression:
invocation-expression
object-creation-expression
assignment
post-increment-expression
post-decrement-expression
pre-increment-expression
pre-decrement-expression

Not all expressions are permitted as statements. [Note: In particular, expressions such as $x + y$ and $x == 1$, that merely compute a value (which will be discarded), are not permitted as statements. *end note*]

Execution of an expression statement evaluates the contained expression and then transfers control to the end point of the expression statement. The end point of an *expression-statement* is reachable if that *expression-statement* is reachable.

15.7 Selection statements

Selection statements select one of a number of possible statements for execution based on the value of some expression.

selection-statement:
if-statement
switch-statement

15.7.1 The if statement

The *if* statement selects a statement for execution based on the value of a boolean expression.

if-statement:
if (*boolean-expression*) *embedded-statement*
if (*boolean-expression*) *embedded-statement* **else** *embedded-statement*
boolean-expression:
expression

An **else** part is associated with the lexically nearest preceding *if* that is allowed by the syntax. [Example: Thus, an *if* statement of the form

```
if (x) if (y) F(); else G();
```

1 is equivalent to

```

2     if (x) {
3         if (y) {
4             F();
5         }
6         else {
7             G();
8         }
9     }

```

10 *end example]*

11 An `if` statement is executed as follows:

- 12 • The *boolean-expression* (§14.16) is evaluated.
- 13 • If the boolean expression yields `true`, control is transferred to the first embedded statement. When and
- 14 if control reaches the end point of that statement, control is transferred to the end point of the `if` statement.
- 15 • If the boolean expression yields `false` and if an `else` part is present, control is transferred to the
- 16 second embedded statement. When and if control reaches the end point of that statement, control is
- 17 transferred to the end point of the `if` statement.
- 18 • If the boolean expression yields `false` and if an `else` part is not present, control is transferred to the
- 19 end point of the `if` statement.

20 The first embedded statement of an `if` statement is reachable if the `if` statement is reachable and the

21 boolean expression does not have the constant value `false`.

22 The second embedded statement of an `if` statement, if present, is reachable if the `if` statement is reachable

23 and the boolean expression does not have the constant value `true`.

24 The end point of an `if` statement is reachable if the end point of at least one of its embedded statements is

25 reachable. In addition, the end point of an `if` statement with no `else` part is reachable if the `if` statement is

26 reachable and the boolean expression does not have the constant value `true`.

27 15.7.2 The `switch` statement

28 The `switch` statement selects for execution a statement list having an associated switch label that corresponds

29 to the value of the switch expression.

```

30     switch-statement:
31         switch ( expression ) switch-block
32
33     switch-block:
34         { switch-sectionsopt }
35
36     switch-sections:
37         switch-section
38         switch-sections switch-section
39
40     switch-section:
41         switch-labels statement-list
42
43     switch-labels:
44         switch-label
45         switch-labels switch-label
46
47     switch-label:
48         case constant-expression :
49         default :

```

45 A *switch-statement* consists of the keyword `switch`, followed by a parenthesized expression (called the

46 switch expression), followed by a *switch-block*. The *switch-block* consists of zero or more *switch-sections*,

enclosed in braces. Each *switch-section* consists of one or more *switch-labels* followed by a *statement-list* (§15.2.1).

The **governing type** of a `switch` statement is established by the switch expression. If the type of the switch expression is `sbyte`, `byte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `char`, `string`, or an *enum-type*, then that is the governing type of the `switch` statement. Otherwise, exactly one user-defined implicit conversion (§13.4) must exist from the type of the switch expression to one of the following possible governing types: `sbyte`, `byte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `char`, `string`. If no such implicit conversion exists, or if more than one such implicit conversion exists, a compile-time error occurs.

The constant expression of each `case` label must denote a value of a type that is implicitly convertible (§13.1) to the governing type of the `switch` statement. A compile-time error occurs if two or more `case` labels in the same `switch` statement specify the same constant value.

There can be at most one `default` label in a switch statement.

A `switch` statement is executed as follows:

- The switch expression is evaluated and converted to the governing type.
- If one of the constants specified in a `case` label in the same `switch` statement is equal to the value of the switch expression, control is transferred to the statement list following the matched `case` label.
- If none of the constants specified in `case` labels in the same `switch` statement is equal to the value of the switch expression, and if a `default` label is present, control is transferred to the statement list following the `default` label.
- If none of the constants specified in `case` labels in the same `switch` statement is equal to the value of the switch expression, and if no `default` label is present, control is transferred to the end point of the `switch` statement.

If the end point of the statement list of a switch section is reachable, a compile-time error occurs. This is known as the “no fall through” rule. [Example: The example

```
switch (i) {
    case 0:
        CaseZero();
        break;
    case 1:
        CaseOne();
        break;
    default:
        CaseOthers();
        break;
}
```

is valid because no switch section has a reachable end point. Unlike C and C++, execution of a switch section is not permitted to “fall through” to the next switch section, and the example

```
switch (i) {
    case 0:
        CaseZero();
    case 1:
        CaseZeroOrOne();
    default:
        CaseAny();
}
```

results in a compile-time error. When execution of a switch section is to be followed by execution of another switch section, an explicit `goto case` or `goto default` statement must be used:


```

1      switch (i) {
2      case 0:
3          CaseZero();
4          goto case 1;
5      case 1:
6          CaseZeroOrOne();
7          goto default;
8      default:
9          CaseAny();
10         break;
11     }

```

12 *end example]*

13 Multiple labels are permitted in a *switch-section*. [*Example:* The example

```

14     switch (i) {
15     case 0:
16         CaseZero();
17         break;
18     case 1:
19         CaseOne();
20         break;
21     case 2:
22     default:
23         CaseTwo();
24         break;
25     }

```

26 is valid. The example does not violate the “no fall through” rule because the labels `case 2:` and `default:`
 27 are part of the same *switch-section*. *end example]*

28 [*Note:* The “no fall through” rule prevents a common class of bugs that occur in C and C++ when `break`
 29 statements are accidentally omitted. In addition, because of this rule, the switch sections of a `switch`
 30 statement can be arbitrarily rearranged without affecting the behavior of the statement. For example, the
 31 sections of the `switch` statement above can be reversed without affecting the behavior of the statement:

```

32     switch (i) {
33     default:
34         CaseAny();
35         break;
36     case 1:
37         CaseZeroOrOne();
38         goto default;
39     case 0:
40         CaseZero();
41         goto case 1;
42     }

```

43 *end note]*

44 [*Note:* The statement list of a switch section typically ends in a `break`, `goto case`, or `goto default`
 45 statement, but any construct that renders the end point of the statement list unreachable is permitted. For
 46 example, a `while` statement controlled by the boolean expression `true` is known to never reach its end
 47 point. Likewise, a `throw` or `return` statement always transfers control elsewhere and never reaches its end
 48 point. Thus, the following example is valid:

```

49     switch (i) {
50     case 0:
51         while (true) F();
52     case 1:
53         throw new ArgumentException();
54     case 2:
55         return;
56     }

```

57 *end note]*

58 [*Example:* The governing type of a `switch` statement may be the type `string`. For example:

```

1      void DoCommand(string command) {
2          switch (command.ToLower()) {
3              case "run":
4                  DoRun();
5                  break;
6              case "save":
7                  DoSave();
8                  break;
9              case "quit":
10                 DoQuit();
11                 break;
12             default:
13                 InvalidCommand(command);
14                 break;
15         }
16     }

```

17 *end example]*

18 [Note: Like the string equality operators (§14.9.7), the `switch` statement is case sensitive and will execute a
19 given switch section only if the switch expression string exactly matches a `case` label constant. *end note]*

20 When the governing type of a `switch` statement is `string`, the value `null` is permitted as a case label
21 constant.

22 The *statement-lists* of a *switch-block* may contain declaration statements (§15.5). The scope of a local
23 variable or constant declared in a switch block is the switch block.

24 Within a switch block, the meaning of a name used in an expression context must always be the same
25 (§14.5.2.1).

26 The statement list of a given switch section is reachable if the `switch` statement is reachable and at least
27 one of the following is true:

- 28 • The switch expression is a non-constant value.
- 29 • The switch expression is a constant value that matches a `case` label in the switch section.
- 30 • The switch expression is a constant value that doesn't match any `case` label, and the switch section
31 contains the `default` label.
- 32 • A switch label of the switch section is referenced by a reachable `goto case` or `goto default`
33 statement.

34 The end point of a `switch` statement is reachable if at least one of the following is true:

- 35 • The `switch` statement contains a reachable `break` statement that exits the `switch` statement.
- 36 • The `switch` statement is reachable, the switch expression is a non-constant value, and no `default`
37 label is present.
- 38 • The `switch` statement is reachable, the switch expression is a constant value that doesn't match any
39 `case` label, and no `default` label is present.

40 15.8 Iteration statements

41 Iteration statements repeatedly execute an embedded statement.

```

42     iteration-statement:
43         while-statement
44         do-statement
45         for-statement
46         foreach-statement

```

47 15.8.1 The `while` statement

48 The `while` statement conditionally executes an embedded statement zero or more times.

1 *while-statement:*

2 **while** (*boolean-expression*) *embedded-statement*

3 A **while** statement is executed as follows:

- 4 • The *boolean-expression* (§14.16) is evaluated.
- 5 • If the boolean expression yields **true**, control is transferred to the embedded statement. When and if
- 6 control reaches the end point of the embedded statement (possibly from execution of a **continue**
- 7 statement), control is transferred to the beginning of the **while** statement.
- 8 • If the boolean expression yields **false**, control is transferred to the end point of the **while** statement.

9 Within the embedded statement of a **while** statement, a **break** statement (§15.9.1) may be used to transfer

10 control to the end point of the **while** statement (thus ending iteration of the embedded statement), and a

11 **continue** statement (§15.9.2) may be used to transfer control to the end point of the embedded statement

12 (thus performing another iteration of the **while** statement).

13 The embedded statement of a **while** statement is reachable if the **while** statement is reachable and the

14 boolean expression does not have the constant value **false**.

15 The end point of a **while** statement is reachable if at least one of the following is true:

- 16 • The **while** statement contains a reachable **break** statement that exits the **while** statement.
- 17 • The **while** statement is reachable and the boolean expression does not have the constant value **true**.

18 **15.8.2 The do statement**

19 The **do** statement conditionally executes an embedded statement one or more times.

20 *do-statement:*

21 **do** *embedded-statement* **while** (*boolean-expression*) ;

22 A **do** statement is executed as follows:

- 23 • Control is transferred to the embedded statement.
- 24 • When and if control reaches the end point of the embedded statement (possibly from execution of a
- 25 **continue** statement), the *boolean-expression* (§14.16) is evaluated. If the boolean expression yields **true**,
- 26 control is transferred to the beginning of the **do** statement. Otherwise, control is transferred to the end point
- 27 of the **do** statement.

28 Within the embedded statement of a **do** statement, a **break** statement (§15.9.1) may be used to transfer

29 control to the end point of the **do** statement (thus ending iteration of the embedded statement), and a

30 **continue** statement (§15.9.2) may be used to transfer control to the end point of the embedded statement

31 (thus performing another iteration of the **do** statement).

32 The embedded statement of a **do** statement is reachable if the **do** statement is reachable.

33 The end point of a **do** statement is reachable if at least one of the following is true:

- 34 • The **do** statement contains a reachable **break** statement that exits the **do** statement.
- 35 • The end point of the embedded statement is reachable and the boolean expression does not have the
- 36 constant value **true**.

37 **15.8.3 The for statement**

38 The **for** statement evaluates a sequence of initialization expressions and then, while a condition is true,

39 repeatedly executes an embedded statement and evaluates a sequence of iteration expressions.

40 *for-statement:*

41 **for** (*for-initializer_{opt}* ; *for-condition_{opt}* ; *for-iterator_{opt}*) *embedded-statement*

for-initializer:
 local-variable-declaration
 statement-expression-list

for-condition:
 boolean-expression

for-iterator:
 statement-expression-list

statement-expression-list:
 statement-expression
 statement-expression-list , *statement-expression*

The *for-initializer*, if present, consists of either a *local-variable-declaration* (§15.5.1) or a list of *statement-expressions* (§15.6) separated by commas. The scope of a local variable declared by a *for-initializer* starts at the *local-variable-declarator* for the variable and extends to the end of the embedded statement. The scope includes the *for-condition* and the *for-iterator*.

The *for-condition*, if present, must be a *boolean-expression* (§14.16).

The *for-iterator*, if present, consists of a list of *statement-expressions* (§15.6) separated by commas.

A **for** statement is executed as follows:

- If a *for-initializer* is present, the variable initializers or statement expressions are executed in the order they are written. This step is only performed once.
- If a *for-condition* is present, it is evaluated.
- If the *for-condition* is not present or if the evaluation yields **true**, control is transferred to the embedded statement. When and if control reaches the end point of the embedded statement (possibly from execution of a **continue** statement), the expressions of the *for-iterator*, if any, are evaluated in sequence, and then another iteration is performed, starting with evaluation of the *for-condition* in the step above.
- If the *for-condition* is present and the evaluation yields **false**, control is transferred to the end point of the **for** statement.

Within the embedded statement of a **for** statement, a **break** statement (§15.9.1) may be used to transfer control to the end point of the **for** statement (thus ending iteration of the embedded statement), and a **continue** statement (§15.9.2) may be used to transfer control to the end point of the embedded statement (thus executing another iteration of the **for** statement).

The embedded statement of a **for** statement is reachable if one of the following is true:

- The **for** statement is reachable and no *for-condition* is present.
- The **for** statement is reachable and a *for-condition* is present and does not have the constant value **false**.

The end point of a **for** statement is reachable if at least one of the following is true:

- The **for** statement contains a reachable **break** statement that exits the **for** statement.
- The **for** statement is reachable and a *for-condition* is present and does not have the constant value **true**.

15.8.4 The **foreach** statement

The **foreach** statement enumerates the elements of a collection, executing an embedded statement for each element of the collection.

foreach-statement:
 foreach (*type identifier in expression*) *embedded-statement*

The *type* and *identifier* of a `foreach` statement declare the **iteration variable** of the statement. The iteration variable corresponds to a read-only local variable with a scope that extends over the embedded statement. During execution of a `foreach` statement, the iteration variable represents the collection element for which an iteration is currently being performed. A compile-time error occurs if the embedded statement attempts to modify the iteration variable (via assignment or the `++` and `--` operators) or pass the iteration variable as a `ref` or `out` parameter.

The type of the *expression* of a `foreach` statement must be a collection type (as defined below), and an explicit conversion (§13.2) must exist from the element type of the collection to the type of the iteration variable. If *expression* has the value `null`, a `System.NullReferenceException` is thrown.

A type `C` is said to be a **collection type** if it implements the `System.IEnumerable` interface or implements the **collection pattern** by meeting all of the following criteria:

- `C` contains a `public` instance method with the signature `GetEnumerator()`, that returns a *struct-type*, *class-type*, or *interface-type*, which is called `E` in the following text.
- `E` contains a `public` instance method with the signature `MoveNext()` and the return type `bool`.
- `E` contains a `public` instance property named `Current` that permits reading the current value. The type of this property is said to be the **element type** of the collection type.

A type that implements `IEnumerable` is also a collection type, even if it doesn't satisfy the conditions above. (This is possible if it implements `IEnumerable` via private interface implementation.)

The `System.Array` type (§19.1.1) is a collection type, and since all array types derive from `System.Array`, any array type expression is permitted in a `foreach` statement. The order in which `foreach` traverses the elements of an array is as follows: For single-dimensional arrays elements are traversed in increasing index order, starting with index 0 and ending with index `Length - 1`. For multi-dimensional arrays, elements are traversed such that the indices of the rightmost dimension are increased first, then the next left dimension, and so on to the left.

A `foreach` statement of the form:

```
foreach (ElementType element in collection) statement
```

corresponds to one of two possible expansions:

- If the `collection` expression is of a type that implements the collection pattern (as defined above), the expansion of the `foreach` statement is:

```
Enumerator enumerator = (collection).GetEnumerator();
try {
    while (enumerator.MoveNext()) {
        ElementType element = (ElementType)enumerator.Current;
        statement;
    }
} finally {
    IDisposable disposable = enumerator as System.IDisposable;
    if (disposable != null) disposable.Dispose();
}
```

[Note: Significant optimizations of the above are often easily available. If the type `E` implements `System.IDisposable`, then the expression `(enumerator as System.IDisposable)` will always be non-null and the implementation can safely substitute a simple conversion for a possibly more expensive type test. Conversely, if the type `E` is sealed and does not implement `System.IDisposable`, then the expression `(enumerator as System.IDisposable)` will always evaluate to null. In this case, the implementation can safely optimize away the entire finally clause. *end note*]

• Otherwise; the `collection` expression is of a type that implements `System.IEnumerable`, and the expansion of the `foreach` statement is:

```
IEnumerator enumerator =
((System.IEnumerable)(collection)).GetEnumerator();
try {
    while (enumerator.MoveNext()) {
        ElementType element = (ElementType)enumerator.Current;
        statement;
    }
}
finally {
    IDisposable disposable = enumerator as System.IDisposable;
    if (disposable != null) disposable.Dispose();
}
```

In either expansion, the `enumerator` variable is a temporary variable that is inaccessible in, and invisible to, the embedded statement, and the `element` variable is read-only in the embedded statement.

[Example: The following example prints out each value in a two-dimensional array, in element order:

```
using System;
class Test
{
    static void Main() {
        double[,] values = {
            {1.2, 2.3, 3.4, 4.5},
            {5.6, 6.7, 7.8, 8.9}
        };

        foreach (double elementvalue in values)
            Console.WriteLine("{0} ", elementvalue);
    }
}
```

The output produced is as follows:

```
1.2 2.3 3.4 4.5 5.6 6.7 7.8 8.9
```

end example]

15.9 Jump statements

Jump statements unconditionally transfer control.

```
jump-statement:
    break-statement
    continue-statement
    goto-statement
    return-statement
    throw-statement
```

The location to which a jump statement transfers control is called the **target** of the jump statement.

When a jump statement occurs within a block, and the target of that jump statement is outside that block, the jump statement is said to **exit** the block. While a jump statement may transfer control out of a block, it can never transfer control into a block.

Execution of jump statements is complicated by the presence of intervening `try` statements. In the absence of such `try` statements, a jump statement unconditionally transfers control from the jump statement to its target. In the presence of such intervening `try` statements, execution is more complex. If the jump statement exits one or more `try` blocks with associated `finally` blocks, control is initially transferred to the `finally` block of the innermost `try` statement. When and if control reaches the end point of a `finally` block, control is transferred to the `finally` block of the next enclosing `try` statement. This process is repeated until the `finally` blocks of all intervening `try` statements have been executed.

[Example: In the example

```

using System;
class Test
{
    static void Main() {
        while (true) {
            try {
                try {
                    Console.WriteLine("Before break");
                    break;
                }
                finally {
                    Console.WriteLine("Innermost finally block");
                }
            }
            finally {
                Console.WriteLine("Outermost finally block");
            }
        }
        Console.WriteLine("After break");
    }
}

```

the finally blocks associated with two try statements are executed before control is transferred to the target of the jump statement.

The output produced is as follows:

```

Before break
Innermost finally block
Outermost finally block
After break

```

end example]

15.9.1 The break statement

The **break** statement exits the nearest enclosing **switch**, **while**, **do**, **for**, or **foreach** statement.

break-statement:

```
break ;
```

The target of a **break** statement is the end point of the nearest enclosing **switch**, **while**, **do**, **for**, or **foreach** statement. If a **break** statement is not enclosed by a **switch**, **while**, **do**, **for**, or **foreach** statement, a compile-time error occurs.

When multiple **switch**, **while**, **do**, **for**, or **foreach** statements are nested within each other, a **break** statement applies only to the innermost statement. To transfer control across multiple nesting levels, a **goto** statement (§15.9.3) must be used.

A **break** statement cannot exit a **finally** block (§15.10). When a **break** statement occurs within a **finally** block, the target of the **break** statement must be within the same **finally** block; otherwise a compile-time error occurs.

A **break** statement is executed as follows:

- If the **break** statement exits one or more **try** blocks with associated **finally** blocks, control is initially transferred to the **finally** block of the innermost **try** statement. When and if control reaches the end point of a **finally** block, control is transferred to the **finally** block of the next enclosing **try** statement. This process is repeated until the **finally** blocks of all intervening **try** statements have been executed.
- Control is transferred to the target of the **break** statement.

Because a **break** statement unconditionally transfers control elsewhere, the end point of a **break** statement is never reachable.

15.9.2 The continue statement

The `continue` statement starts a new iteration of the nearest enclosing `while`, `do`, `for`, or `foreach` statement.

```
continue-statement:
    continue ;
```

The target of a `continue` statement is the end point of the embedded statement of the nearest enclosing `while`, `do`, `for`, or `foreach` statement. If a `continue` statement is not enclosed by a `while`, `do`, `for`, or `foreach` statement, a compile-time error occurs.

When multiple `while`, `do`, `for`, or `foreach` statements are nested within each other, a `continue` statement applies only to the innermost statement. To transfer control across multiple nesting levels, a `goto` statement (§15.9.3) must be used.

A `continue` statement cannot exit a `finally` block (§15.10). When a `continue` statement occurs within a `finally` block, the target of the `continue` statement must be within the same `finally` block; otherwise a compile-time error occurs.

A `continue` statement is executed as follows:

- If the `continue` statement exits one or more `try` blocks with associated `finally` blocks, control is initially transferred to the `finally` block of the innermost `try` statement. When and if control reaches the end point of a `finally` block, control is transferred to the `finally` block of the next enclosing `try` statement. This process is repeated until the `finally` blocks of all intervening `try` statements have been executed.
- Control is transferred to the target of the `continue` statement.

Because a `continue` statement unconditionally transfers control elsewhere, the end point of a `continue` statement is never reachable.

15.9.3 The goto statement

The `goto` statement transfers control to a statement that is marked by a label.

```
goto-statement:
    goto identifier ;
    goto case constant-expression ;
    goto default ;
```

The target of a `goto identifier` statement is the labeled statement with the given label. If a label with the given name does not exist in the current function member, or if the `goto` statement is not within the scope of the label, a compile-time error occurs. [Note: This rule permits the use of a `goto` statement to transfer control *out of* a nested scope, but not *into* a nested scope. In the example

```
using System;
class Test
{
    static void Main(string[] args) {
        string[,] table = {
            {"red", "blue", "green"},
            {"Monday", "Wednesday", "Friday"}
        };
        foreach (string str in args) {
            int row, colm;
            for (row = 0; row <= 1; ++row)
                for (colm = 0; colm <= 2; ++colm)
                    if (str == table[row, colm])
                        goto done;
        }
    }
}
```



```

1         Console.WriteLine("{0} not found", str);
2         continue;
3     done:
4         Console.WriteLine("Found {0} at [{1}][{2}]", str, row, colm);
5     }
6 }
7

```

8 a **goto** statement is used to transfer control out of a nested scope. *end note*

9 The target of a **goto case** statement is the statement list in the immediately enclosing switch statement (§15.7.2) which contains a **case** label with the given constant value. If the **goto case** statement is not enclosed by a **switch** statement, if the *constant-expression* is not implicitly convertible (§13.1) to the governing type of the nearest enclosing **switch** statement, or if the nearest enclosing **switch** statement does not contain a **case** label with the given constant value, a compile-time error occurs.

14 The target of a **goto default** statement is the statement list in the immediately enclosing switch statement (§15.7.2), which contains a **default** label. If the **goto default** statement is not enclosed by a **switch** statement, or if the nearest enclosing **switch** statement does not contain a **default** label, a compile-time error occurs.

18 A **goto** statement cannot exit a **finally** block (§15.10). When a **goto** statement occurs within a **finally** block, the target of the **goto** statement must be within the same **finally** block, or otherwise a compile-time error occurs.

21 A **goto** statement is executed as follows:

- 22 • If the **goto** statement exits one or more **try** blocks with associated **finally** blocks, control is initially transferred to the **finally** block of the innermost **try** statement. When and if control reaches the end point of a **finally** block, control is transferred to the **finally** block of the next enclosing **try** statement. This process is repeated until the **finally** blocks of all intervening **try** statements have been executed.
- 26 • Control is transferred to the target of the **goto** statement.

27 Because a **goto** statement unconditionally transfers control elsewhere, the end point of a **goto** statement is never reachable.

29 15.9.4 The return statement

30 The **return** statement returns control to the caller of the function member in which the **return** statement appears.

```

32     return-statement:
33         return expressionopt ;

```

34 A **return** statement with no expression can be used only in a function member that does not compute a value; that is, a method with the return type **void**, the **set** accessor of a property or indexer, the add and remove accessors of an event, an instance constructor, static constructor, or a destructor.

37 A **return** statement with an expression can only be used in a function member that computes a value, that is, a method with a non-void return type, the **get** accessor of a property or indexer, or a user-defined operator. An implicit conversion (§13.1) must exist from the type of the expression to the return type of the containing function member.

41 It is a compile-time error for a **return** statement to appear in a **finally** block (§15.10).

42 A **return** statement is executed as follows:

- 43 • If the **return** statement specifies an expression, the expression is evaluated and the resulting value is converted to the return type of the containing function member by an implicit conversion. The result of the conversion becomes the value returned to the caller.
- 46 • If the **return** statement is enclosed by one or more **try** blocks with associated **finally** blocks, control is initially transferred to the **finally** block of the innermost **try** statement. When and if control

reaches the end point of a **finally** block, control is transferred to the **finally** block of the next enclosing **try** statement. This process is repeated until the **finally** blocks of all enclosing **try** statements have been executed.

- Control is returned to the caller of the containing function member.

Because a **return** statement unconditionally transfers control elsewhere, the end point of a **return** statement is never reachable.

15.9.5 The **throw** statement

The **throw** statement throws an exception.

```
throw-statement:
    throw expressionopt ;
```

A **throw** statement with an expression throws the value produced by evaluating the expression. The expression must denote a value of the class type **System.Exception** or of a class type that derives from **System.Exception**. If evaluation of the expression produces **null**, a **System.NullReferenceException** is thrown instead.

A **throw** statement with no expression can be used only in a **catch** block, in which case, that statement re-throws the exception that is currently being handled by that **catch** block.

Because a **throw** statement unconditionally transfers control elsewhere, the end point of a **throw** statement is never reachable.

When an exception is thrown, control is transferred to the first **catch** clause in an enclosing **try** statement that can handle the exception. The process that takes place from the point of the exception being thrown to the point of transferring control to a suitable exception handler is known as **exception propagation**.

Propagation of an exception consists of repeatedly evaluating the following steps until a **catch** clause that matches the exception is found. In this description, the **throw point** is initially the location at which the exception is thrown.

- In the current function member, each **try** statement that encloses the throw point is examined. For each statement **S**, starting with the innermost **try** statement and ending with the outermost **try** statement, the following steps are evaluated:
 - If the **try** block of **S** encloses the throw point and if **S** has one or more **catch** clauses, the **catch** clauses are examined in order of appearance to locate a suitable handler for the exception. The first **catch** clause that specifies the exception type or a base type of the exception type is considered a match. A general **catch** (§15.10) clause is considered a match for any exception type. If a matching **catch** clause is located, the exception propagation is completed by transferring control to the block of that **catch** clause.
 - Otherwise, if the **try** block or a **catch** block of **S** encloses the throw point and if **S** has a **finally** block, control is transferred to the **finally** block. If the **finally** block throws another exception, processing of the current exception is terminated. Otherwise, when control reaches the end point of the **finally** block, processing of the current exception is continued.

- If an exception handler was not located in the current function member invocation, the function member invocation is terminated. The steps above are then repeated for the caller of the function member with a throw point corresponding to the statement from which the function member was invoked.
- If the exception processing terminates all function member invocations in the current thread, indicating that the thread has no handler for the exception, then the thread is itself terminated. The impact of such termination is implementation-defined.

15.10 The try statement

The `try` statement provides a mechanism for catching exceptions that occur during execution of a block. Furthermore, the `try` statement provides the ability to specify a block of code that is always executed when control leaves the `try` statement.

```

try-statement:
    try block catch-clauses
    try block finally-clause
    try block catch-clauses finally-clause

catch-clauses:
    specific-catch-clauses general-catch-clauseopt
    specific-catch-clausesopt general-catch-clause

specific-catch-clauses:
    specific-catch-clause
    specific-catch-clauses specific-catch-clause

specific-catch-clause:
    catch ( class-type identifieropt ) block

general-catch-clause:
    catch block

finally-clause:
    finally block

```

There are three possible forms of `try` statements:

- A `try` block followed by one or more `catch` blocks.
- A `try` block followed by a `finally` block.
- A `try` block followed by one or more `catch` blocks followed by a `finally` block.

When a `catch` clause specifies a *class-type*, the type must be `System.Exception` or a type that derives from `System.Exception`.

When a `catch` clause specifies both a *class-type* and an *identifier*, an **exception variable** of the given name and type is declared. The exception variable corresponds to a local variable with a scope that extends over the `catch` block. During execution of the `catch` block, the exception variable represents the exception currently being handled. For purposes of definite assignment checking, the exception variable is considered definitely assigned in its entire scope.

Unless a `catch` clause includes an exception variable name, it is impossible to access the exception object in the `catch` block.

A `catch` clause that specifies neither an exception type nor an exception variable name is called a general `catch` clause. A `try` statement can only have one general `catch` clause, and if one is present it must be the last `catch` clause.

[Note: Some environments, especially those supporting multiple languages, may support exceptions that are not representable as an object derived from `System.Exception`, although such an exception could never be generated by C# code. In such an environment, a general `catch` clause might be used to catch such an

exception. Thus, a general catch clause is semantically different from one that specifies the type `System.Exception`, in that the former may also catch exceptions from other languages. *end note*

In order to locate a handler for an exception, `catch` clauses are examined in lexical order. A compile-time error occurs if a `catch` clause specifies a type that is the same as, or is derived from, a type that was specified in an earlier `catch` clause for the same `try`. [*Note*: Without this restriction, it would be possible to write unreachable `catch` clauses. *end note*]

Within a `catch` block, a `throw` statement (§15.9.5) with no expression can be used to re-throw the exception that was caught by the `catch` block. Assignments to an exception variable do not alter the exception that is re-thrown.

[*Example*: In the example

```
using System;
class Test
{
    static void F() {
        try {
            G();
        }
        catch (Exception e) {
            Console.WriteLine("Exception in F: " + e.Message);
            e = new Exception("F");
            throw;           // re-throw
        }
    }
    static void G() {
        throw new Exception("G");
    }
    static void Main() {
        try {
            F();
        }
        catch (Exception e) {
            Console.WriteLine("Exception in Main: " + e.Message);
        }
    }
}
```

the method `F` catches an exception, writes some diagnostic information to the console, alters the exception variable, and re-throws the exception. The exception that is re-thrown is the original exception, so the output produced is:

```
Exception in F: G
Exception in Main: G
```

If the first `catch` block had thrown `e` instead of rethrowing the current exception, the output produced would be as follows:

```
Exception in F: G
Exception in Main: F
```

end example]

It is a compile-time error for a `break`, `continue`, or `goto` statement to transfer control out of a `finally` block. When a `break`, `continue`, or `goto` statement occurs in a `finally` block, the target of the statement must be within the same `finally` block, or otherwise a compile-time error occurs.

It is a compile-time error for a `return` statement to occur in a `finally` block.

A `try` statement is executed as follows:

- Control is transferred to the `try` block.
- When and if control reaches the end point of the `try` block:
 - o If the `try` statement has a `finally` block, the `finally` block is executed.

- o Control is transferred to the end point of the `try` statement.
- If an exception is propagated to the `try` statement during execution of the `try` block:
 - o The `catch` clauses, if any, are examined in order of appearance to locate a suitable handler for the exception. The first `catch` clause that specifies the exception type or a base type of the exception type is considered a match. A general `catch` clause is considered a match for any exception type. If a matching `catch` clause is located:
 - If the matching `catch` clause declares an exception variable, the exception object is assigned to the exception variable.
 - Control is transferred to the matching `catch` block.
 - When and if control reaches the end point of the `catch` block:
 - o If the `try` statement has a `finally` block, the `finally` block is executed.
 - o Control is transferred to the end point of the `try` statement.
 - If an exception is propagated to the `try` statement during execution of the `catch` block:
 - o If the `try` statement has a `finally` block, the `finally` block is executed.
 - o The exception is propagated to the next enclosing `try` statement.
 - o If the `try` statement has no `catch` clauses or if no `catch` clause matches the exception:
 - If the `try` statement has a `finally` block, the `finally` block is executed.
 - The exception is propagated to the next enclosing `try` statement.

The statements of a `finally` block are always executed when control leaves a `try` statement. This is true whether the control transfer occurs as a result of normal execution, as a result of executing a `break`, `continue`, `goto`, or `return` statement, or as a result of propagating an exception out of the `try` statement.

If an exception is thrown during execution of a `finally` block, the exception is propagated to the next enclosing `try` statement. If another exception was in the process of being propagated, that exception is lost. The process of propagating an exception is discussed further in the description of the `throw` statement (§15.9.5).

The `try` block of a `try` statement is reachable if the `try` statement is reachable.

A `catch` block of a `try` statement is reachable if the `try` statement is reachable.

The `finally` block of a `try` statement is reachable if the `try` statement is reachable.

The end point of a `try` statement is reachable if both of the following are true:

- The end point of the `try` block is reachable or the end point of at least one `catch` block is reachable.
- If a `finally` block is present, the end point of the `finally` block is reachable.

15.11 The checked and unchecked statements

The `checked` and `unchecked` statements are used to control the *overflow checking context* for integral-type arithmetic operations and conversions.

checked-statement:

`checked block`

unchecked-statement:

`unchecked block`

The `checked` statement causes all expressions in the *block* to be evaluated in a checked context, and the `unchecked` statement causes all expressions in the *block* to be evaluated in an unchecked context.

The **checked** and **unchecked** statements are precisely equivalent to the **checked** and **unchecked** operators (§14.5.12), except that they operate on blocks instead of expressions.

15.12 The lock statement

The **lock** statement obtains the mutual-exclusion lock for a given object, executes a statement, and then releases the lock.

```
lock-statement:
    lock ( expression ) embedded-statement
```

The expression of a **lock** statement must denote a value of a *reference-type*. No implicit boxing conversion (§13.1.5) is ever performed for the expression of a **lock** statement, and thus it is a compile-time error for the expression to denote a value of a *value-type*.

A **lock** statement of the form

```
lock (x) ...
```

where *x* is an expression of a *reference-type*, is precisely equivalent to

```
System.Threading.Monitor.Enter(x);
try {
    ...
}
finally {
    System.Threading.Monitor.Exit(x);
}
```

except that *x* is only evaluated once.

[*Example:* The `System.Type` object of a class can conveniently be used as the mutual-exclusion lock for static methods of the class. For example:

```
class Cache
{
    public static void Add(object x) {
        lock (typeof(Cache)) {
            ...
        }
    }

    public static void Remove(object x) {
        lock (typeof(Cache)) {
            ...
        }
    }
}
```

end example]

15.13 The using statement

The **using** statement obtains one or more resources, executes a statement, and then disposes of the resource.

```
using-statement:
    using ( resource-acquisition ) embedded-statement
```

```
resource-acquisition:
    local-variable-declaration
    expression
```

A **resource** is a class or struct that implements `System.IDisposable`, which includes a single parameterless method named `Dispose`. Code that is using a resource can call `Dispose` to indicate that the resource is no longer needed. If `Dispose` is not called, then automatic disposal eventually occurs as a consequence of garbage collection.

1 If the form of *resource-acquisition* is *local-variable-declaration* then the type of the *local-variable-*
 2 *declaration* must be `System.IDisposable` or a type that can be implicitly converted to
 3 `System.IDisposable`. If the form of *resource-acquisition* is *expression* then this expression must be
 4 `System.IDisposable` or a type that can be implicitly converted to `System.IDisposable`.

5 Local variables declared in a *resource-acquisition* are read-only, and must include an initializer. A compile-
 6 time error occurs if the embedded statement attempts to modify these local variables (via assignment or the
 7 ++ and -- operators) or pass them as `ref` or `out` parameters.

8 A `using` statement is translated into three parts: acquisition, usage, and disposal. Usage of the resource is
 9 implicitly enclosed in a `try` statement that includes a `finally` clause. This `finally` clause disposes of the
 10 resource. If a `null` resource is acquired, then no call to `Dispose` is made, and no exception is thrown.

11 A `using` statement of the form

```
12     using (R r1 = new R()) {
13         r1.F();
14     }
```

15 is precisely equivalent to

```
16     R r1 = new R();
17     try {
18         r1.F();
19     }
20     finally {
21         if (r1 != null) ((IDisposable)r1).Dispose();
22     }
```

23 A *resource-acquisition* may acquire multiple resources of a given type. This is equivalent to nested `using`
 24 statements. A `using` statement of the form

```
25     using (R r1 = new R(), r2 = new R()) {
26         r1.F();
27         r2.F();
28     }
```

29 is precisely equivalent to:

```
30     using (R r1 = new R())
31     using (R r2 = new R()) {
32         r1.F();
33         r2.F();
34     }
```

35 which is, by expansion, precisely equivalent to:

```
36     R r1 = new R();
37     try {
38         R r2 = new R();
39         try {
40             r1.F();
41             r2.F();
42         }
43         finally {
44             if (r2 != null) ((IDisposable)r2).Dispose();
45         }
46     }
47     finally {
48         if (r1 != null) ((IDisposable)r1).Dispose();
49     }
```


16. Namespaces

C# programs are organized using namespaces. Namespaces are used both as an “internal” organization system for a program, and as an “external” organization system—a way of presenting program elements that are exposed to other programs.

Using-directives (§16.3) are provided to facilitate the use of namespaces.

16.1 Compilation units

A *compilation-unit* defines the overall structure of a source file. A compilation unit consists of zero or more *using-directives* followed by zero or more *global-attributes* followed by zero or more *namespace-member-declarations*.

compilation-unit:

*using-directives*_{opt} *global-attributes*_{opt} *namespace-member-declarations*_{opt}

A C# program consists of one or more compilation units, each contained in a separate source file. When a C# program is compiled, all of the compilation units are processed together. Thus, compilation units can depend on each other, possibly in a circular fashion.

The *using-directives* of a compilation unit affect the *global-attributes* and *namespace-member-declarations* of that compilation unit, but have no effect on other compilation units.

The *global-attributes* (§24) of a compilation unit permit the specification of attributes for the target assembly. Assemblies act as physical containers for types.

The *namespace-member-declarations* of each compilation unit of a program contribute members to a single declaration space called the global namespace. [Example: For example:

File A.cs:

```
class A {}
```

File B.cs:

```
class B {}
```

The two compilation units contribute to the single global namespace, in this case declaring two classes with the fully qualified names A and B. Because the two compilation units contribute to the same declaration space, it would have been an error if each contained a declaration of a member with the same name. *end example*]

16.2 Namespace declarations

A *namespace-declaration* consists of the keyword `namespace`, followed by a namespace name and body, optionally followed by a semicolon.

namespace-declaration:

`namespace` *qualified-identifier* *namespace-body* ;_{opt}

qualified-identifier:

identifier

qualified-identifier . *identifier*

namespace-body:

{ *using-directives*_{opt} *namespace-member-declarations*_{opt} }

A *namespace-declaration* may occur as a top-level declaration in a *compilation-unit* or as a member declaration within another *namespace-declaration*. When a *namespace-declaration* occurs as a top-level declaration in a *compilation-unit*, the namespace becomes a member of the global namespace. When a *namespace-declaration* occurs within another *namespace-declaration*, the inner namespace becomes a member of the outer namespace. In either case, the name of a namespace must be unique within the containing namespace.

Namespaces are implicitly `public` and the declaration of a namespace cannot include any access modifiers.

Within a *namespace-body*, the optional *using-directives* import the names of other namespaces and types, allowing them to be referenced directly instead of through qualified names. The optional *namespace-member-declarations* contribute members to the declaration space of the namespace. Note that all *using-directives* must appear before any member declarations.

The *qualified-identifier* of a *namespace-declaration* may be a single identifier or a sequence of identifiers separated by “.” tokens. The latter form permits a program to define a nested namespace without lexically nesting several namespace declarations. [Example: For example,

```
namespace N1.N2
{
    class A {}
    class B {}
}
```

is semantically equivalent to

```
namespace N1
{
    namespace N2
    {
        class A {}
        class B {}
    }
}
```

end example]

Namespaces are open-ended, and two namespace declarations with the same fully qualified name contribute to the same declaration space (§10.3). [Example: In the example

```
namespace N1.N2
{
    class A {}
}

namespace N1.N2
{
    class B {}
}
```

the two namespace declarations above contribute to the same declaration space, in this case declaring two classes with the fully qualified names `N1.N2.A` and `N1.N2.B`. Because the two declarations contribute to the same declaration space, it would have been an error if each contained a declaration of a member with the same name. *end example*]

16.3 Using directives

Using-directives facilitate the use of namespaces and types defined in other namespaces. *Using-directives* impact the name resolution process of *namespace-or-type-names* (§10.8) and *simple-names* (§14.5.2), but unlike declarations, *using-directives* do not contribute new members to the underlying declaration spaces of the compilation units or namespaces within which they are used.

```
using-directives:
    using-directive
    using-directives using-directive
```

```

1      using-directive:
2          using-alias-directive
3          using-namespace-directive

```

4 A *using-alias-directive* (§16.3.1) introduces an alias for a namespace or type.

5 A *using-namespace-directive* (§16.3.2) imports the type members of a namespace.

6 The scope of a *using-directive* extends over the *namespace-member-declarations* of its immediately
 7 containing compilation unit or namespace body. The scope of a *using-directive* specifically does not include
 8 its peer *using-directives*. Thus, peer *using-directives* do not affect each other, and the order in which they are
 9 written is insignificant.

10 16.3.1 Using alias directives

11 A *using-alias-directive* introduces an identifier that serves as an alias for a namespace or type within the
 12 immediately enclosing compilation unit or namespace body.

```

13      using-alias-directive:
14          using identifier = namespace-or-type-name ;

```

15 Within member declarations in a compilation unit or namespace body that contains a *using-alias-directive*,
 16 the identifier introduced by the *using-alias-directive* can be used to reference the given namespace or type.
 17 [Example: For example:

```

18      namespace N1.N2
19      {
20          class A {}
21      }
22      namespace N3
23      {
24          using A = N1.N2.A;
25          class B: A {}
26      }

```

27 Above, within member declarations in the N3 namespace, A is an alias for N1.N2.A, and thus class N3.B
 28 derives from class N1.N2.A. The same effect can be obtained by creating an alias R for N1.N2 and then
 29 referencing R.A:

```

30      namespace N3
31      {
32          using R = N1.N2;
33          class B: R.A {}
34      }

```

35 *end example]*

36 The *identifier* of a *using-alias-directive* must be unique within the declaration space of the compilation unit
 37 or namespace that immediately contains the *using-alias-directive*. [Example: For example:

```

38      namespace N3
39      {
40          class A {}
41      }
42      namespace N3
43      {
44          using A = N1.N2.A;      // Error, A already exists
45      }

```

46 Above, N3 already contains a member A, so it is a compile-time error for a *using-alias-directive* to use that
 47 identifier. *end example]* Likewise, it is a compile-time error for two or more *using-alias-directives* in the
 48 same compilation unit or namespace body to declare aliases by the same name.

49 A *using-alias-directive* makes an alias available within a particular compilation unit or namespace body, but
 50 it does not contribute any new members to the underlying declaration space. In other words, a *using-alias-*

directive is not transitive, but, rather, affects only the compilation unit or namespace body in which it occurs.

[*Example:* In the example

```

1      namespace N3
2      {
3          using R = N1.N2;
4      }
5
6      namespace N3
7      {
8          class B: R.A {}           // Error, R unknown
9      }
10

```

the scope of the *using-alias-directive* that introduces *R* only extends to member declarations in the namespace body in which it is contained, so *R* is unknown in the second namespace declaration. However, placing the *using-alias-directive* in the containing compilation unit causes the alias to become available within both namespace declarations:

```

15     using R = N1.N2;
16
17     namespace N3
18     {
19         class B: R.A {}
20     }
21
22     namespace N3
23     {
24         class C: R.A {}
25     }

```

end example]

Just like regular members, names introduced by *using-alias-directives* are hidden by similarly named members in nested scopes. [*Example:* In the example

```

27     using R = N1.N2;
28
29     namespace N3
30     {
31         class R {}
32         class B: R.A {}           // Error, R has no member A
33     }

```

the reference to *R.A* in the declaration of *B* causes a compile-time error because *R* refers to *N3.R*, not *N1.N2*. *end example]*

The order in which *using-alias-directives* are written has no significance, and resolution of the *namespace-or-type-name* referenced by a *using-alias-directive* is not affected by the *using-alias-directive* itself or by other *using-directives* in the immediately containing compilation unit or namespace body. In other words, the *namespace-or-type-name* of a *using-alias-directive* is resolved as if the immediately containing compilation unit or namespace body had no *using-directives*. [*Example:* In the example

```

40     namespace N1.N2 {}
41
42     namespace N3
43     {
44         using R1 = N1;           // OK
45         using R2 = N1.N2;       // OK
46         using R3 = R1.N2;       // Error, R1 unknown
47     }

```

the last *using-alias-directive* results in a compile-time error because it is not affected by the first *using-alias-directive*. *end example]*

A *using-alias-directive* can create an alias for any namespace or type, including the namespace within which it appears and any namespace or type nested within that namespace.

Accessing a namespace or type through an alias yields exactly the same result as accessing that namespace or type through its declared name. [Example: For example, given

```

namespace N1.N2
{
    class A {}
}

namespace N3
{
    using R1 = N1;
    using R2 = N1.N2;

    class B
    {
        N1.N2.A a;           // refers to N1.N2.A
        R1.N2.A b;           // refers to N1.N2.A
        R2.A c;              // refers to N1.N2.A
    }
}

```

the names `N1.N2.A`, `R1.N2.A`, and `R2.A` are equivalent and all refer to the class whose fully qualified name is `N1.N2.A`. end example]

16.3.2 Using namespace directives

A *using-namespace-directive* imports the types contained in a namespace into the immediately enclosing compilation unit or namespace body, enabling the identifier of each type to be used without qualification.

```

using-namespace-directive:
    using namespace-name ;

```

Within member declarations in a compilation unit or namespace body that contains a *using-namespace-directive*, the types contained in the given namespace can be referenced directly. [Example: For example:

```

namespace N1.N2
{
    class A {}
}

namespace N3
{
    using N1.N2;

    class B: A {}
}

```

Above, within member declarations in the `N3` namespace, the type members of `N1.N2` are directly available, and thus class `N3.B` derives from class `N1.N2.A`. end example]

A *using-namespace-directive* imports the types contained in the given namespace, but specifically does not import nested namespaces. [Example: In the example

```

namespace N1.N2
{
    class A {}
}

namespace N3
{
    using N1;

    class B: N2.A {}    // Error, N2 unknown
}

```

the *using-namespace-directive* imports the types contained in `N1`, but not the namespaces nested in `N1`. Thus, the reference to `N2.A` in the declaration of `B` results in a compile-time error because no members named `N2` are in scope. end example]

Unlike a *using-alias-directive*, a *using-namespace-directive* may import types whose identifiers are already defined within the enclosing compilation unit or namespace body. In effect, names imported by a *using-*

namespace-directive are hidden by similarly named members in the enclosing compilation unit or namespace body. [Example: For example:

```

namespace N1.N2
{
    class A {}
    class B {}
}
namespace N3
{
    using N1.N2;
    class A {}
}
```

Here, within member declarations in the N3 namespace, A refers to N3.A rather than N1.N2.A. *end example*]

When more than one namespace imported by *using-namespace-directives* in the same compilation unit or namespace body contain types by the same name, references to that name are considered ambiguous.

[Example: In the example

```

namespace N1
{
    class A {}
}
namespace N2
{
    class A {}
}
namespace N3
{
    using N1;
    using N2;
    class B: A {}           // Error, A is ambiguous
}
```

both N1 and N2 contain a member A, and because N3 imports both, referencing A in N3 is a compile-time error. *end example*] In this situation, the conflict can be resolved either through qualification of references to A, or by introducing a *using-alias-directive* that picks a particular A. [Example: For example:

```

namespace N3
{
    using N1;
    using N2;
    using A = N1.A;
    class B: A {}           // A means N1.A
}
```

end example]

Like a *using-alias-directive*, a *using-namespace-directive* does not contribute any new members to the underlying declaration space of the compilation unit or namespace, but, rather, affects only the compilation unit or namespace body in which it appears.

The *namespace-name* referenced by a *using-namespace-directive* is resolved in the same way as the *namespace-or-type-name* referenced by a *using-alias-directive*. Thus, *using-namespace-directives* in the same compilation unit or namespace body do not affect each other and can be written in any order.

16.4 Namespace members

A *namespace-member-declaration* is either a *namespace-declaration* (§16.2) or a *type-declaration* (§16.5).

```

1      namespace-member-declarations:
2          namespace-member-declaration
3      namespace-member-declarations namespace-member-declaration
4
5      namespace-member-declaration:
6          namespace-declaration
7          type-declaration

```

A compilation unit or a namespace body can contain *namespace-member-declarations*, and such declarations contribute new members to the underlying declaration space of the containing compilation unit or namespace body.

16.5 Type declarations

A *type-declaration* is a *class-declaration* (§17.1), a *struct-declaration* (§18.1), an *interface-declaration* (§20.1), an *enum-declaration* (§21.1), or a *delegate-declaration* (§22.1).

```

13     type-declaration:
14         class-declaration
15         struct-declaration
16         interface-declaration
17         enum-declaration
18         delegate-declaration

```

A *type-declaration* can occur as a top-level declaration in a compilation unit or as a member declaration within a namespace, class, or struct.

When a type declaration for a type *T* occurs as a top-level declaration in a compilation unit, the fully qualified name of the newly declared type is simply *T*. When a type declaration for a type *T* occurs within a namespace, class, or struct, the fully qualified name of the newly declared type is *N.T*, where *N* is the fully qualified name of the containing namespace, class, or struct.

A type declared within a class or struct is called a nested type (§17.2.6).

The permitted access modifiers and the default access for a type declaration depend on the context in which the declaration takes place (§10.5.1):

- Types declared in compilation units or namespaces can have `public` or `internal` access. The default is `internal` access.
- Types declared in classes can have `public`, `protected internal`, `protected`, `internal`, or `private` access. The default is `private` access.
- Types declared in structs can have `public`, `internal`, or `private` access. The default is `private` access.

